ASSIGNMENT 2 – CO2

Functions and Object-Oriented Concepts in C++

Bloom's Level K5 – Evaluate | Total Marks: 50 | Individual Assignment

---

Item	Details
Name	__Hemalatha.C__
Register Number	__192524475__
Submission Date	31/10/2026
GitHub Repository	https://github.com/Hemalatha-1806/DSA0102-C-
Compiler	g++ 13.3.0 (Ubuntu), Linux x86-64
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file
Timing runs	The same command with -O0 or -O2 added (Q6 and Q7)

How this report is organised

For every question I give the program, the prediction I made before running it, the real output, and the explanation. Predictions are shown before the outputs. The main programs were compiled with `-Wall -Wextra` and gave no warning; the only warning and errors in this report are the ones shown on purpose in Q2 and Q3. Timings are from my own machine, so the exact milliseconds will be different on another computer; the comparison between versions is what matters.

Summary of results (README note for each task)

Task	Folder	Result
Q1	Q1	Predicted output 1 12 13 / 1 100 / 16 13 6 was correct. Only call by reference and call by pointer change the stored fee (45000 → 40500). Design chosen: return the new fee. Complete-the-code output matches the expected output.

Task	Folder	Result
Q2	Q2	Returning int& lets +2 moderation change the array element directly. Returning a reference to a local gives warning -Wreturn-local-addr and crashes with a segmentation fault when used. A vector buffer moved after push_back, so an old reference becomes invalid.
Q3	Q3	Original code gives "call of overloaded charge(double) is ambiguous". Redesign with int / double / Parcel overloads and defaults on one function per type. All 6 predicted selections were correct; charge(2) selects the grams version.
Q4	Q4	SavingsAccount keeps the last 10 transactions in a private array (members defined inside and outside the class). 12 transactions were made and only the last 10 are shown. A circular buffer prints the same statement.
Q5	Q5	Token prediction 101 103 104 4 was correct. BusTicket output matches expected (Seat 1, 2, 3, Waiting, Issued: 3). The static counter keeps its value across function calls.
Q6	Q6	Friend functions give 205 false. All six correctness tests pass. At -O2 the inline version is about 13% faster than a forced non-inline call; at -O0 inline makes no difference.
Q7	Q7	Redesigned the Q4 shifting array as a circular buffer. Recording time became independent of capacity; 5.6× to 1913× faster at -O0 depending on capacity, identical statement, no extra memory (padding absorbed the new int).

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Folder: Q1 Files: predict.cpp, three_ways.cpp, complete.cpp

1.1 Predict the Output

My prediction (written before running)

```
1 12 13
1 100
16 13 6
```

How I got it: change(a, b, &c) – a is passed by value, so only a copy becomes 11 and the real a stays 1. b is a reference, so the real b becomes 12. c is passed as an address and *c += 10 changes the real c to 13. In bigger(a, c) = 100 the values are 1 and 13, so the function returns a reference to c and the assignment writes 100 into c. Finally the three show() calls print 16, 13 and 6, each followed by a space.

Program :

```cpp
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}

```

Actual output

```
1 12 13
1 100
16 13 6
```

Difference between prediction and output: none. The last line ends with a space after the 6 because `show()` prints `' '` after every number, and the `'\n'` comes only afterwards from `main()`.

Which variables changed in each call

Call	a	b	c	Reason
`change(a, b, &c)`	1 (no change)	2 → 12	3 → 13	a: by value (copy). b: reference (alias). c: pointer, changed through `*c`.
`bigger(a, c) = 100`	1	12	13 → 100	Returns `int&` to the larger one (c), so the assignment changes c itself.

Default arguments used by each call to show()

Call	x, y, z used	Defaults used	Calculation
`show(1)`	1, 5, 10	y = 5 and z = 10	1 + 5 + 10 = 16
`show(1, 2)`	1, 2, 10	z = 10 only	1 + 2 + 10 = 13
`show(1, 2, 3)`	1, 2, 3	none	1 + 2 + 3 = 6

1.2 Hostel Fee Concession – three versions

The fee is 45,000 and the concession is 10% (merit). Before running I predicted: **by value → stored fee stays 45000, by reference → 40500, by pointer → 40500.**

```cpp
#include <iostream>
#include <string>
// 1. by value  -> works on a copy, caller's fee never changes
void concessionByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "  inside byValue   : " << fee << '\n';
}

// 2. by reference -> fee is an alias of the caller's variable
void concessionByRef(long long& fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "  inside byRef     : " << fee << '\n';
}

// 3. by pointer -> pointer holds the caller's address, we dereference it
void concessionByPtr(long long* fee, int percent)
{
    if (fee == nullptr) return;           // pointers can be null, so check
    *fee -= *fee * percent / 100;
    std::cout << "  inside byPtr     : " << *fee << '\n';
}

// Design decision: return the new fee (pure function), caller decides what to store
long long concessionReturn(long long fee, int percent)
{
    return fee - fee * percent / 100;
}

// const reference: read only, no copy of a big object
struct Student {
    std::string name;
    long long fee;
    char notes[1000];                     // makes the object large
};
void printStudent(const Student& s)
{
    std::cout << "  " << s.name << " fee = " << s.fee << '\n';
}

int main()
{
    long long fee = 45000;

    std::cout << "Start fee = " << fee << "\n\n";

    std::cout << "Call by value (10% merit):\n";
    concessionByValue(fee, 10);
    std::cout << "  stored fee after : " << fee << "\n\n";

    std::cout << "Call by reference (10% merit):\n";
    concessionByRef(fee, 10);
    std::cout << "  stored fee after : " << fee << "\n\n";

    fee = 45000;
    std::cout << "Call by pointer (10% merit):\n";
    concessionByPtr(&fee, 10);
    std::cout << "  stored fee after : " << fee << "\n\n";

    // several concessions on one student: merit 10 + sports 15 + single parent 25
    fee = 45000;
    long long f = concessionReturn(concessionReturn(concessionReturn(fee, 10), 15), 25);
    std::cout << "Chained return version: 45000 -> " << f << '\n';

    Student s{"Asha", 45000, {}};
    std::cout << "\nconst reference print:\n";
    printStudent(s);
    std::cout << "sizeof(long long) = " << sizeof(long long)
              << ", sizeof(Student) = " << sizeof(Student) << '\n';
}
```

Observed output

```
Start fee = 45000

Call by value (10% merit):
  inside byValue   : 40500
  stored fee after : 45000

Call by reference (10% merit):
  inside byRef     : 40500
  stored fee after : 40500

Call by pointer (10% merit):
  inside byPtr     : 40500
  stored fee after : 40500

Chained return version: 45000 -> 25819

const reference print:
  Asha fee = 45000
sizeof(long long) = 8, sizeof(Student) = 1040
```

The prediction was correct. Note that the by-value function also prints 40500 inside, which is exactly why the bug looked like "it works" – the print inside the function shows the local copy, not the stored fee.

1.3 Inquiry

Where each version stores its parameter

Version	Where the parameter lives	Caller's fee changed?
By value	A new local variable (8 bytes) in the function's stack frame, holding a copy of the value.	No
By reference	No new fee variable. The name fee is just another name (alias) for the caller's variable.	Yes

Version	Where the parameter lives	Caller's fee changed?
By pointer	A new local pointer variable (8 bytes on 64-bit) holding the address of the caller's variable. The fee itself stays in the caller.	Yes, through *fee

Only reference and pointer let the function reach the caller's original memory. In call by value the function works on its own copy, and that copy is destroyed when the function returns, so the change is lost.

When to use a const reference: when the function only needs to read an object and the object is big or expensive to copy (a Student with strings and arrays, a vector, a std::string). `const Student&` avoids the copy and the compiler stops the function from modifying it. For small types like `int` or `long long` plain by value is just as cheap, so a const reference gives no benefit there.

1.4 Design Decision – in place or return the new fee?

My choice: return the new fee. The signature is `[[nodiscard]] long long applyConcession(long long fee, int percent)` and the caller writes `fee = applyConcession(fee, 10);`.

- Several concessions can be chained: `applyConcession(applyConcession(applyConcession(fee, 10), 15), 25)` gives 25819 for 45000. No temporary variables are needed.
- The function has no side effects, so the office can preview the fee before saving it, and it is easy to test.
- The data flow is visible at the call site. The original bug happened because nothing in the call showed that the result was being thrown away. With `[[nodiscard]]` the compiler warns if the caller forgets to store it.

One more thing I noticed: because integer division truncates, the *order* of concessions changes the answer. 10% → 15% → 25% gives 25819, but 25% → 15% → 10% gives 25820 (I checked both by hand and by running). So the office must fix one order in the rules. The in-place reference version would be better only when updating one big record again and again and the copy cost matters, which is not the case for a single long long.

1.5 Complete the Code

TODO 1 is filled with the same formula, but on the reference parameter so that the caller's variable is changed. Nothing else in the program was changed.

```cpp
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;   // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
    // TODO 1 (done): reduce the caller's fee by percent using integer arithmetic
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}

```

Output (matches the expected output)

```
after call by value : 45000
after call by reference: 40500
```

Why call by value leaves 45000: `applyByValue` receives its own copy of `hostelFee`. The line `fee -= fee * percent / 100` reduces that copy to 40500, and the copy is destroyed when the function ends. `hostelFee` in `main` was never touched.

1.6 Alternative Concept and Complexity Analysis

Instead of changing the fee through a reference, the function can return the new fee (this is the version from 1.4):

```
long long applyConcession(long long fee, int percent)
{
   return fee - fee * percent / 100;
}

// caller
hostelFee = applyConcession(hostelFee, 10);
```

Time is O(1) for all versions: one multiplication, one division and one subtraction. What differs is the amount of data copied per call (measured with `sizeof` in my program: `long long` = 8 bytes, my test `Student` with a 1000-character notes array = 1040 bytes):

Passing style	**long long (8 B)**	**Student (1040 B)**	**Space complexity**
By value	8 bytes copied	1040 bytes copied every call	O(1) for long long, O(s) for an object of size s
By reference	nothing copied (address only)	nothing copied (address only)	O(1)
By const reference	nothing copied	nothing copied, and read-only	O(1)
Return new fee	8 bytes returned	not suitable for big objects	O(1) for long long

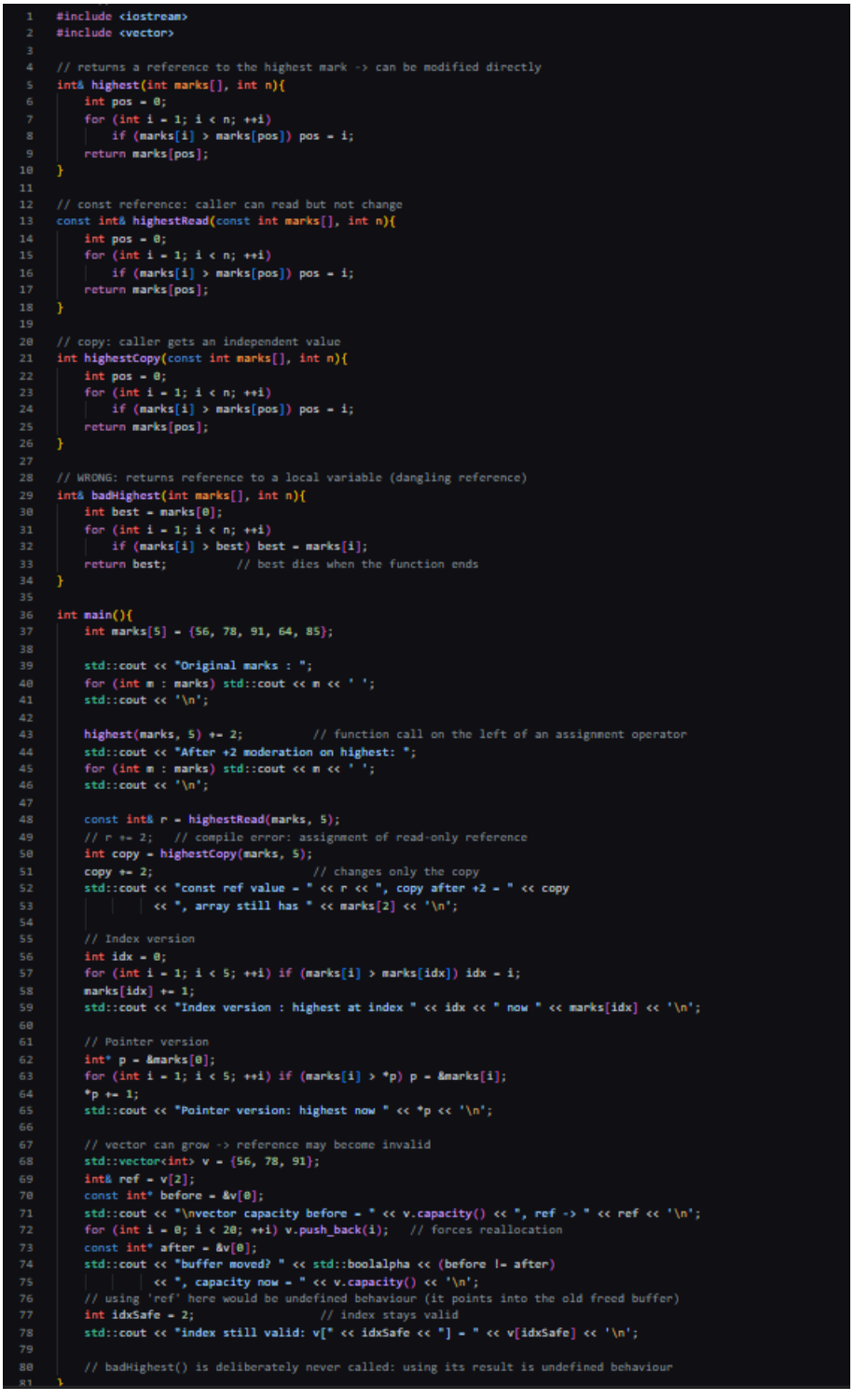
So for a single fee the cost of copying is negligible and returning the value is the safest design. For a large Student object, a const reference is the right way to read it and a non-const reference the right way to modify it.

Q2 – Return by Reference and Its Dangers (4 Marks)

Folder: Q2 Files: marks.cpp, bad_call.cpp, warning.txt

2.1 Working functions

highest() returns int& to the element with the largest mark. Because it is a reference to the array element itself, highest(marks, 5) += 2; applies the +2 moderation directly to the array, with the call on the left side of the assignment operator. (In Q1, bigger(a, c) = 100 used plain = in the same way.)



```cpp
#include <iostream>
#include <vector>

// returns a reference to the highest mark -> can be modified directly
int& highest(int marks[], int n){
    int pos = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[pos]) pos = i;
    return marks[pos];
}

// const reference: caller can read but not change
const int& highestRead(const int marks[], int n){
    int pos = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[pos]) pos = i;
    return marks[pos];
}

// copy: caller gets an independent value
int highestCopy(const int marks[], int n){
    int pos = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[pos]) pos = i;
    return marks[pos];
}

// WRONG: returns reference to a local variable (dangling reference)
int& badHighest(int marks[], int n){
    int best = marks[0];
    for (int i = 1; i < n; ++i)
        if (marks[i] > best) best = marks[i];
    return best;          // best dies when the function ends
}

int main(){
    int marks[5] = {56, 78, 91, 64, 85};

    std::cout << "Original marks : ";
    for (int m : marks) std::cout << m << ' ';
    std::cout << '\n';

    highest(marks, 5) += 2;          // function call on the left of an assignment operator
    std::cout << "After +2 moderation on highest: ";
    for (int m : marks) std::cout << m << ' ';
    std::cout << '\n';

    const int& r = highestRead(marks, 5);
    // r += 2;   // compile error: assignment of read-only reference
    int copy = highestCopy(marks, 5);
    copy += 2;                       // changes only the copy
    std::cout << "const ref value = " << r << ", copy after +2 = " << copy
              << ", array still has " << marks[2] << '\n';

    // Index version
    int idx = 0;
    for (int i = 1; i < 5; ++i) if (marks[i] > marks[idx]) idx = i;
    marks[idx] += 1;
    std::cout << "Index version : highest at index " << idx << " now " << marks[idx] << '\n';

    // Pointer version
    int* p = &marks[0];
    for (int i = 1; i < 5; ++i) if (marks[i] > *p) p = &marks[i];
    *p += 1;
    std::cout << "Pointer version: highest now " << *p << '\n';

    // vector can grow -> reference may become invalid
    std::vector<int> v = {56, 78, 91};
    int& ref = v[2];
    const int* before = &v[0];
    std::cout << "\nvector capacity before = " << v.capacity() << ", ref -> " << ref << '\n';
    for (int i = 0; i < 20; ++i) v.push_back(i);   // forces reallocation
    const int* after = &v[0];
    std::cout << "buffer moved? " << std::boolalpha << (before != after)
              << ", capacity now = " << v.capacity() << '\n';
    // using 'ref' here would be undefined behaviour (it points into the old freed buffer)
    int idxSafe = 2;                 // index stays valid
    std::cout << "index still valid: v[" << idxSafe << "] = " << v[idxSafe] << '\n';

    // badHighest() is deliberately never called: using its result is undefined behaviour
}
```

Output

```
Original marks : 56 78 91 64 85
After +2 moderation on highest: 56 78 93 64 85
const ref value = 93, copy after +2 = 95, array still has 93
Index version : highest at index 2 now 94
Pointer version: highest now 95

vector capacity before = 3, ref -> 91
buffer moved? true, capacity now = 24
index still valid: v[2] = 91
```

The marks 56 78 91 64 85 became 56 78 93 64 85 – only the highest mark (91) was changed. The "capacity now = 24" value depends on the standard library implementation; the important part is that the buffer moved.

2.2 Returning a reference to a local variable

The function badHighest() stores the best mark in a local variable best and returns a reference to it. Compiling with g++ -std=c++17 -Wall -Wextra gives this warning:

```
marks.cpp: In function 'int& badHighest(int*, int)':
marks.cpp:37:12: warning: reference to local variable 'best' returned [-Wreturn-local-addr]
   37 |     return best;          // best dies when the function ends
      |            ^~~~
marks.cpp:34:9: note: declared here
   34 |     int best = marks[0];
      |         ^~~~
```

The reason is that best lives on the function's stack frame and is destroyed when the function returns. The caller receives a reference to memory that is no longer valid – a dangling reference. Reading or writing through it is **undefined behaviour**: the program may print garbage, may seem to work, or may crash. To see this I wrote bad_call.cpp which uses the returned reference:

```cpp
#include <iostream>
int& badHighest(int marks[], int n)
{
    int best = marks[0];
    for (int i = 1; i < n; ++i) if (marks[i] > best) best = marks[i];
    return best;
}
int main()
{
    int marks[5] = {56, 78, 91, 64, 85};
    int& r = badHighest(marks, 5);
    std::cout << "value through dangling reference: " << r << '\n';
}
```

Result on my machine (both -O0 and -O2)

> Segmentation fault (exit code 139)

GCC does not even give the address of the dead local variable here; it returns a null reference, so the program crashes. On another compiler or version the result can differ, and that is exactly the problem with undefined behaviour.

2.3 Reference vs const reference vs copy

Return type	What the caller gets	Can the caller change the array?	Notes
`int&`	The element itself (alias)	Yes,	Can be used on the left of =. Dangerous if the object dies.
`const int&`	The element itself, read only	No (compile error on r += 2)	No copy, no accidental change. Still dangling if the array is destroyed.
`int`	An independent copy	No (only the copy changes)	Always safe. Copy is cheap for int. Used in the program: copy became 95 but array stayed 93.

2.4 Changed Condition – the register is a std::vector that can grow

A std::vector keeps its elements in one block of memory. When `push_back` needs more room than the current capacity, the vector allocates a bigger block, copies (moves) the elements and frees the old block. A reference, pointer or iterator taken earlier still points into the old block, so it is dangling. In my test the reference `ref = v[2]` was taken when capacity was 3; after 20 push_backs the buffer address changed (`buffer moved? true`) so using `ref` would be undefined behaviour. I did not use it after that point on purpose.

Safe ways: take the reference only after all insertions are finished, use `reserve()` beforehand so the vector never reallocates, or keep an index and look up `v[i]` each time.

2.5 Alternative Concept and Complexity Analysis

Returning an index or a pointer instead of a reference:

```
int highestIndex(const int marks[], int n)   // returns -1 if n == 0
{
   int pos = 0;
   for (int i = 1; i < n; ++i) if (marks[i] > marks[pos]) pos = i;
   return n > 0 ? pos : -1;
}
int* highestPtr(int marks[], int n)          // returns nullptr if n == 0
{
   int* best = marks;
   for (int i = 1; i < n; ++i) if (marks[i] > *best) best = &marks[i];
   return n > 0 ? best : nullptr;
}

// calling code
marks[highestIndex(marks, 5)] += 2;      // index version
*highestPtr(marks, 5) += 2;              // pointer version
highest(marks, 5) += 2;                  // reference version
```

All three scan the array once, so the time is **O(n)** and the extra space is **O(1)** (one int or one pointer). The difference is safety:

Version	Empty array (n = 0)	After the vector grows	Verdict
int& (reference)	Cannot report "not found"; a reference must refer to something	Dangling → undefined behaviour	Cleanest syntax, least safe
int* (pointer)	Can return nullptr (caller must check)	Dangling, same as reference	Can signal failure, but same lifetime risk
int (index)	Can return -1	Still valid, as long as no elements are erased or inserted before it	Safest when the container can grow

So for a fixed array the reference version is fine and very convenient. Once the register becomes a growing vector, I would return the index.

Q3 – Default Arguments and Function Overloading (6 Marks)

Folder: Q3 Files: ambiguous.cpp, courier.cpp, long_call.cpp, alt_struct.cpp

3.1 The compiler error

```cpp
#include <iostream>

double charge(double weight)                    { return 50 + 30 * weight; }
double charge(double weight, bool express)      { return 50 + 30 * weight + (express ? 50 : 0); }
double charge(double weight, bool express = false, bool insured = false)
{
    return 50 + 30 * weight + (express ? 50 : 0) + (insured ? 20 : 0);
}

int main()
{
    std::cout << charge(2.0) << '\n';        // which one?
    std::cout << charge(2.0, true) << '\n';  // which one?
}

```

```
ambiguous.cpp: In function 'int main()':
ambiguous.cpp:12:24: error: call of overloaded 'charge(double)' is ambiguous
   12 |     std::cout << charge(2.0) << '\n';        // which one?
      |                  ~~~~~~^~~~~
ambiguous.cpp:3:8: note: candidate: 'double charge(double)'
    3 | double charge(double weight)                    { return 50 + 30 * weight; }
      |        ^~~~~~
ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
ambiguous.cpp:13:24: error: call of overloaded 'charge(double, bool)' is ambiguous
   13 |     std::cout << charge(2.0, true) << '\n';  // which one?
      |                  ~~~~~~^~~~~~~~~~~
ambiguous.cpp:4:8: note: candidate: 'double charge(double, bool)'
    4 | double charge(double weight, bool express)      { return 50 + 30 * weight + (express ? 50 : 0); }
      |        ^~~~~~
ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
    5 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
```

For charge(2.0) both charge(double) and charge(double, bool = false, bool = false) are viable and both are an exact match on the one argument given. The defaulted parameters do not make the second one a worse match, so the compiler cannot choose and reports an ambiguous call. In the same way charge(2.0, true) matches both charge(double, bool) and the three-parameter version. The root cause is that the same call can be written in several ways because the default arguments overlap with the other overloads.

3.2 Redesign

Rule I followed: only *one* function per kind of input carries the default arguments, and the default arguments are written only in the declaration. The three kinds of input are grams (int), kilograms (double) and a Parcel object.

```cpp
#include <iostream>

struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};

// default arguments only on ONE function per kind of input -> no ambiguity
double charge(int grams,  bool express = false, bool insured = false);
double charge(double kg,  bool express = false, bool insured = false);
double charge(const Parcel& p);

double charge(double kg, bool express, bool insured)
{
    std::cout << "[double kg version] ";
    return 50 + 30 * kg + (express ? 50 : 0) + (insured ? 20 : 0);
}

double charge(int grams, bool express, bool insured)
{
    std::cout << "[int grams version] ";
    // not calling charge(double...) here so the message stays clear
    double kg = grams / 1000.0;
    return 50 + 30 * kg + (express ? 50 : 0) + (insured ? 20 : 0);
}

double charge(const Parcel& p)
{
    std::cout << "[Parcel version]    ";
    return 50 + 30 * p.weightKg + (p.express ? 50 : 0) + (p.insured ? 20 : 0);
}

int main()
{
    Parcel p{3.0, true, true};

    std::cout << "1. charge(500)          = " << charge(500) << '\n';
    std::cout << "2. charge(2.5)          = " << charge(2.5) << '\n';
    std::cout << "3. charge(2)            = " << charge(2) << '\n';
    std::cout << "4. charge(2.0f)         = " << charge(2.0f) << '\n';
    std::cout << "5. charge(1.5, true)    = " << charge(1.5, true) << '\n';
    std::cout << "6. charge(p)            = " << charge(p) << '\n';
    std::cout << "7. charge(1500,true,true) = " << charge(1500, true, true) << '\n';
}
```

My predictions (before execution) and what was observed

No.	Call	Predicted function	Reason
1	charge(500)	int grams	500 is an int → exact match
2	charge(2.5)	double kg	2.5 is a double → exact match
3	charge(2)	int grams	2 is an int. It is 2 grams, not 2 kg (50.06)!

No.	Call	Predicted function	Reason
4	charge(2.0f)	double kg	float → double is a promotion; float → int is a conversion. Promotion wins.
5	charge(1.5, true)	double kg, express	double exact; true matches bool
6	charge(p)	Parcel	Parcel exact match

Observed output

```
1. charge(500)          = [int grams version] 65
2. charge(2.5)          = [double kg version] 125
3. charge(2)            = [int grams version] 50.06
4. charge(2.0f)         = [double kg version] 110
5. charge(1.5, true)    = [double kg version] 145
6. charge(p)            = [Parcel version]    210
7. charge(1500,true,true) = [int grams version] 165
```

All predictions matched. Call 3 is the trap: `charge(2)` silently chooses the grams version, so a user who means 2 kg gets a charge of 50.06. This is a weakness of overloading on unit by type. It can be avoided by giving the functions different names, or by using small wrapper types such as `Grams` and `Kilograms`. Another trap: `charge(2L)` (a long) does not compile because long → int and long → double are both conversions of equal rank:

```
int main() { std::cout << charge(2L) << '\n'; }
// error: call of overloaded 'charge(long int)' is ambiguous
```

The output order of the labels and the "[... version]" tags is correct only from C++17 onwards, where the left operand of `<<` is evaluated first, so I compile with `-std=c++17`.

3.3 Inquiry – how the compiler chooses

The compiler first collects all functions with that name, then keeps those that can accept the arguments (number of arguments, and a conversion exists). Among these it ranks each argument conversion from best to worst:

- **Exact match** – same type (also adding const, array-to-pointer, reference binding).
- **Promotion** – float → double, char/short/bool → int.

- **Standard conversion** – int → double, double → int, long → int, etc.
- **User-defined conversion** – through a converting constructor or conversion operator.

The function that is better on at least one argument and not worse on any other wins. If no function is clearly best, the call is ambiguous. The return type is never used in this decision.

Why default arguments must be given from the right: in a call the arguments are matched to parameters by position from the left, and you cannot skip a middle argument. If a middle parameter had a default but a later one did not, the default could never be used. So `void f(int a = 1, int b);` is a compile error, while `void f(int a, int b = 2, int c = 3);` is legal.

3.4 Alternative Concept and Complexity Analysis

Instead of overloads with default arguments, one struct with optional fields, or one function template:

```cpp
#include <iostream>
#include <type_traits>

// Alternative 1: one struct with optional fields (defaults live in the struct)
struct ChargeRequest {
    double kg;
    bool express = false;
    bool insured = false;
};
double charge(const ChargeRequest& r)
{
    return 50 + 30 * r.kg + (r.express ? 50 : 0) + (r.insured ? 20 : 0);
}

// Alternative 2: function template, unit decided at compile time
template <class W>
double chargeT(W weight, bool express = false, bool insured = false)
{
    double kg = std::is_integral<W>::value ? weight / 1000.0 : double(weight);
    return 50 + 30 * kg + (express ? 50 : 0) + (insured ? 20 : 0);
}

int main()
{
    std::cout << charge({2.0}) << ' ' << charge({2.0, true}) << ' ' << charge({2.0, true, true}) << '\n';
    std::cout << chargeT(500) << ' ' << chargeT(2.5) << ' ' << chargeT(2.0f, true) << '\n';
}
```

Output

```
110 160 180
65 125 160
```

	Overloads + defaults	**Struct with optional fields**	**Function template**
Time	Overload resolution at compile time, no run-time cost. Function itself O(1).	Same, O(1)	Same, O(1) (resolved at compile time)
Space copied per call	A few bytes (a double/int and two bools, in registers)	Passed by const reference: nothing copied (16 bytes if by value)	Same as overloads; one extra copy of the function body per type (code size, not run time)
Adding a new option (e.g. fragile)	Add another defaulted parameter to every overload; calls like charge(2.0, false, true, false) become unreadable	Add one field with a default; no existing call changes	Add the parameter to the template; same readability problem as overloads
Ambiguity risk	High (as seen)	None	Medium (type deduced, e.g. charge(2) means grams)

For three optional things the overload design is fine and short to call. If more options are expected, the struct approach grows best.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

Folder: Q4 Files: account_shift.cpp, account_circular.cpp

4.1 Class specification

Private data: account number, balance, an array of 10 Transaction records and a count. Public functions: deposit(), withdraw() and miniStatement(). getBalance() and deposit() are defined inside the class (implicitly inline). withdraw(), miniStatement() and the private helper record() are defined outside with SavingsAccount:: (scope resolution operator). Older transactions are removed by shifting the array left.



```cpp
#include <iostream>
#include <iomanip>

struct Transaction {
    char   type;        // 'D' deposit, 'W' withdrawal
    double amount;
    double balanceAfter;
};

class SavingsAccount {
    static const int MAX = 10;          // change to 50 for the bank's new requirement
    long        accNo;
    double      balance;
    Transaction history[MAX];           // private: only the class may touch the order
    int         count;                  // number of valid entries (0..MAX)

    void record(char type, double amount);   // helper, defined outside

public:
    SavingsAccount(long no, double opening) : accNo(no), balance(opening), count(0) {}

    // defined inside the class (implicitly inline)
    double getBalance() const { return balance; }

    void deposit(double amount)
    {
        if (amount <= 0) return;
        balance += amount;
        record('D', amount);
    }

    // defined outside using the scope resolution operator
    bool withdraw(double amount);
    void miniStatement() const;
};

void SavingsAccount::record(char type, double amount)
{
    Transaction t{type, amount, balance};
    if (count == MAX) {                     // 11th transaction: drop the oldest
        for (int i = 0; i < MAX - 1; ++i)   // shift left by one  -> O(MAX)
            history[i] = history[i + 1];
        history[MAX - 1] = t;
    } else {
        history[count++] = t;
    }
}

bool SavingsAccount::withdraw(double amount)
{
    if (amount <= 0 || amount > balance) return false;
    balance -= amount;
    record('W', amount);
    return true;
}

void SavingsAccount::miniStatement() const
{
    std::cout << "Mini statement for A/c " << accNo << '\n';
    std::cout << std::fixed << std::setprecision(2);
    for (int i = 0; i < count; ++i)
        std::cout << std::setw(2) << i + 1 << ". "
                  << (history[i].type == 'D' ? "Deposit   " : "Withdrawal")
                  << std::setw(10) << history[i].amount
                  << "   Bal " << std::setw(10) << history[i].balanceAfter << '\n';
}

int main()
{
    SavingsAccount acc(1001234, 5000);
    for (int i = 1; i <= 7; ++i) acc.deposit(100 * i);       // 7 transactions
    acc.withdraw(250);
    acc.withdraw(100);
    acc.deposit(1000);
    acc.withdraw(75);                                        // 11th
    acc.deposit(20);                                         // 12th
    std::cout << "12 transactions done, only the last 10 are kept\n\n";
    acc.miniStatement();
    std::cout << "\nBalance = " << acc.getBalance() << '\n';
}
```

Output (12 transactions made, only the last 10 are shown)

```
12 transactions done, only the last 10 are kept

Mini statement for A/c 1001234
 1. Deposit       300.00   Bal    5600.00
 2. Deposit       400.00   Bal    6000.00
 3. Deposit       500.00   Bal    6500.00
 4. Deposit       600.00   Bal    7100.00
 5. Deposit       700.00   Bal    7800.00
 6. Withdrawal    250.00   Bal    7550.00
 7. Withdrawal    100.00   Bal    7450.00
 8. Deposit      1000.00   Bal    8450.00
 9. Withdrawal     75.00   Bal    8375.00
10. Deposit        20.00   Bal    8395.00

Balance = 8395.00
```

The first two deposits (100 and 200) were dropped when the 11th and 12th transactions arrived, so the statement starts at the 300 deposit. The balance is the real balance, because `balance` is updated separately from the history.

4.2 Inquiry

Why the array must be private. The class depends on two rules: `count` is never more than 10, and the array is in order from oldest to newest. If the array and count were public, any code could write `count = 15` and then `miniStatement()` would read past the end of the array (undefined behaviour), or change an old transaction without changing the balance. Keeping it private means only the member functions can change it, so the rules always hold. It also lets me change the inside later (for example to a circular buffer, see Q7) without touching any code that uses the class.

What happens at the 11th transaction.

Method	What happens	Cost per new transaction	Space
Shift the array	Every element moves one place left (9 copies of a 24-byte record for k = 10), the oldest falls out, the new one goes in the last slot.	O(k)	Fixed array of k records
Circular index	The new transaction overwrites the slot of the oldest one, and the start index moves by one (with wrap-around). Nothing is copied.	O(1)	Fixed array of k records + one int (start)

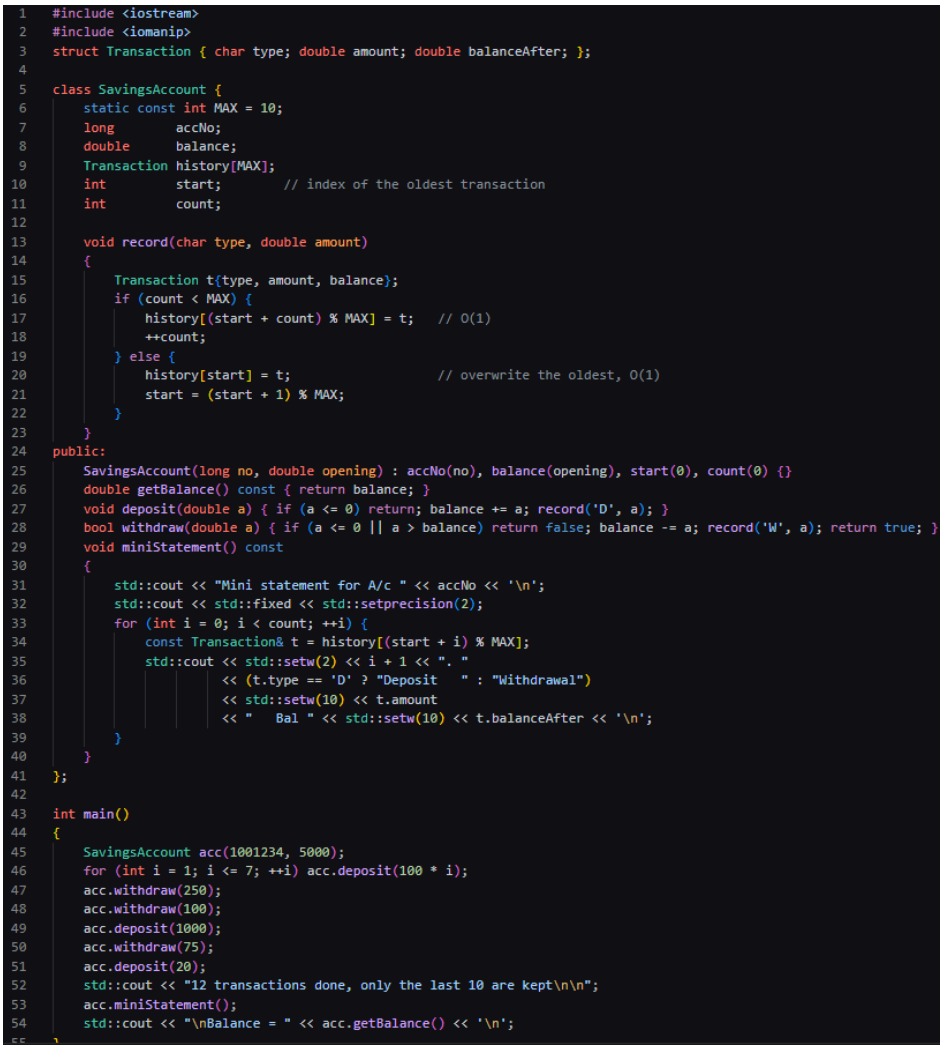
Reading the statement costs O(k) in both designs, because all k entries are printed.

4.3 Design Decision – last 50 transactions

The circular-index design changes least. In both designs the size is only the constant MAX (change 10 to 50), because every loop and the array use MAX and nothing else is hard-coded. But the cost behind that change is different. With shifting, one transaction now copies 49 records instead of 9, so recording becomes about five times more work (my measurement in Q7: 21.6 ms → 92.2 ms for 1,000,000 transactions at -O0). With the circular index the cost per transaction stays the same, and the code of deposit(), withdraw() and miniStatement() does not change at all. Memory for each account grows from 264 to 1224 bytes in both designs; that is a cost of the requirement, not of the design.

4.4 Alternative Concept and Complexity Analysis

Circular buffer version of the class (full program in account_circular.cpp); it prints exactly the same statement as the shifting version, which I confirmed with diff:



```cpp
#include <iostream>
#include <iomanip>
struct Transaction { char type; double amount; double balanceAfter; };

class SavingsAccount {
    static const int MAX = 10;
    long        accNo;
    double      balance;
    Transaction history[MAX];
    int         start;        // index of the oldest transaction
    int         count;

    void record(char type, double amount)
    {
        Transaction t{type, amount, balance};
        if (count < MAX) {
            history[(start + count) % MAX] = t;   // O(1)
            ++count;
        } else {
            history[start] = t;                   // overwrite the oldest, O(1)
            start = (start + 1) % MAX;
        }
    }
public:
    SavingsAccount(long no, double opening) : accNo(no), balance(opening), start(0), count(0) {}
    double getBalance() const { return balance; }
    void deposit(double a) { if (a <= 0) return; balance += a; record('D', a); }
    bool withdraw(double a) { if (a <= 0 || a > balance) return false; balance -= a; record('W', a); return true; }
    void miniStatement() const
    {
        std::cout << "Mini statement for A/c " << accNo << '\n';
        std::cout << std::fixed << std::setprecision(2);
        for (int i = 0; i < count; ++i) {
            const Transaction& t = history[(start + i) % MAX];
            std::cout << std::setw(2) << i + 1 << ". "
                      << (t.type == 'D' ? "Deposit   " : "Withdrawal")
                      << std::setw(10) << t.amount
                      << "   Bal " << std::setw(10) << t.balanceAfter << '\n';
        }
    }
};

int main()
{
    SavingsAccount acc(1001234, 5000);
    for (int i = 1; i <= 7; ++i) acc.deposit(100 * i);
    acc.withdraw(250);
    acc.withdraw(100);
    acc.deposit(1000);
    acc.withdraw(75);
    acc.deposit(20);
    std::cout << "12 transactions done, only the last 10 are kept\n\n";
    acc.miniStatement();
    std::cout << "\nBalance = " << acc.getBalance() << '\n';
}
```

With `std::deque` the code is simply `h.push_back(t); if (h.size() > MAX) h.pop_front();`.

Design	**Add a transaction**	**Space**	**Comments**
Shift fixed array	O(k)	O(k) fixed	Simplest code and the array is always in order. Time grows with k.
Circular buffer	O(1)	O(k) fixed + one int	Fastest. Index arithmetic (start + i) % MAX is needed when reading.
std::deque	O(1) amortised	O(k) + block bookkeeping	No manual index code and size can change, but it allocates memory on the heap and is slower than the buffer in my test.

Measured timings for all three are in Q7.

Q5 – Static Members and Arrays of Objects (7 Marks)

Folder: Q5 Files: token.cpp, busticket.cpp, busticket_function.cpp, busticket_big.cpp, ticketoffice.cpp

5.1 Predict the Output

My prediction (written before running)

> 101 103 104 4

Reasoning: next starts at 100. The array of 3 creates tokens with ids 101, 102, 103, then extra gets 104. So counter[0].get() = 101, counter[2].get() = 103, extra.get() = 104, and issued() = 104 − 100 = 4.

```cpp
#include <iostream>

class Token {
    static int next;
    int id;

public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
              << ' ' << Token::issued() << '\n';
}
```

Actual output

> 101 103 104 4

No difference. The static counter starts at 100 because of the line int Token::next = 100; outside the class. A static data member has static storage duration and it is initialised once, before main() starts, not by any constructor. The constructor ran 4 times (3 for the array elements plus 1 for extra), and each run did ++next on the same shared variable.

5.2 From the principle to a class design

Principle: data that belongs to the whole class should exist only once. For the bus tickets that gives:

- **Static data members:** issued (seats given so far) and CAPACITY (seats in the bus). They describe the bus, not one ticket, so there must be one copy only.
- **Non-static members:** seat and name – every ticket has its own.
- **Static member function:** count() – the transport office can call BusTicket::count() without any ticket object.

Where a static data member is defined. Inside the class it is only declared. It must be defined exactly once outside the class in one source file: int BusTicket::issued = 0;. Without it the program compiles but fails at link time with "undefined reference". (static const int CAPACITY = 3; is an exception: an integral constant with an initialiser can be used directly, because it is only used as a value here.)

Why a static member function cannot use this or a non-static member. this is the address of the object a member function was called on. A static function is called on the class (BusTicket::count()) and there may be no object at all, so there is no this and no seat or name that it could mean. It can only use static members. Writing static int getSeat() { return seat; } gives the error "invalid use of member in static member function".

How the array is created. BusTicket has only a constructor that needs a name, and it is explicit, so there is no default constructor. Therefore BusTicket trip[4]; would not compile; every element must be initialised: BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"), ...};. The constructors run in order of the index, so the seats are given in the order of the list.

What happens to the count in a loop or function. The counter belongs to the class, not to any object, so destroying objects does not reduce it. Tickets created inside a function or loop increase issued each time, and the value is still there after the function returns (shown in 5.4).

5.3 Complete the Code

TODO 1: if issued < CAPACITY, increase issued and take it as the seat number, otherwise seat stays 0 (waiting list). TODO 2: return issued.

```cpp
#include <iostream>
#include <string>

class BusTicket {
    static int issued;                  // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;

public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1 (done)
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }                               // else seat stays 0 -> waiting list
    }

    static int count()
    {
        // TODO 2 (done)
        return issued;
    }

    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;              // definition of the static data member

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Output (matches the expected output)

> Asha: Seat 1
>
> Bala: Seat 2
>
> Chitra: Seat 3
>
> Dev: Waiting
>
> Issued: 3

5.4 Test Your Design

Array larger than the capacity (6 tickets, 3 seats):

> Asha: Seat 1
>
> Bala: Seat 2
>
> Chitra: Seat 3
>
> Dev: Waiting

```
Eswar: Waiting
Fathima: Waiting
Issued: 3
```

Tickets created inside a function that is called three times (two tickets each time, in busticket_function.cpp):

```cpp
void bookTwo(const std::string& a, const std::string& b)
{
    BusTicket t1(a), t2(b);             // destroyed when the function ends
    t1.print();
    t2.print();
}

int main()
{
    std::cout << "Call 1:\n";
    bookTwo("Asha", "Bala");
    std::cout << "  issued after call 1 = " << BusTicket::count() << '\n';
    std::cout << "Call 2:\n";
    bookTwo("Chitra", "Dev");
    std::cout << "  issued after call 2 = " << BusTicket::count() << '\n';
    std::cout << "Call 3:\n";
    bookTwo("Eswar", "Fathima");
    std::cout << "  issued after call 3 = " << BusTicket::count() << '\n';
}
```

```
Call 1:
  Asha: Seat 1
  Bala: Seat 2
  issued after call 1 = 2
Call 2:
  Chitra: Seat 3
  Dev: Waiting
  issued after call 2 = 3
Call 3:
  Eswar: Waiting
  Fathima: Waiting
  issued after call 3 = 3
```

The local tickets are destroyed at the end of every call, but issued stays at 2, then 3, then 3. Seat 3 was given in the second call, and from then on everybody goes to the waiting list. One consequence of this design: if a passenger cancels and the ticket object is destroyed, the seat is not given back, because nothing decrements the static counter. A real system would need a destructor or a cancel function for that.

5.5 Alternative Concept and Complexity Analysis

Seat counter in a separate `TicketOffice` object passed to every ticket (the global variable version would look the same, with the counter outside any class):

```cpp
#include <iostream>
#include <string>

class TicketOffice {            // one office object per bus
    int issued = 0;
    int capacity;
public:
    explicit TicketOffice(int cap) : capacity(cap) {}
    int nextSeat() { return issued < capacity ? ++issued : 0; }   // 0 = waiting
    int count() const { return issued; }
};

class BusTicket {
    int seat;
    std::string name;
public:
    BusTicket(const std::string& n, TicketOffice& office) : seat(office.nextSeat()), name(n) {}
    void print() const
    {
        std::cout << name << ": " << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
    }
};

int main()
{
    TicketOffice bus1(3), bus2(2);          // two buses at the same time
    BusTicket a("Asha", bus1), b("Bala", bus1), c("Chitra", bus2), d("Dev", bus2), e("Eswar", bus2);
    a.print(); b.print(); c.print(); d.print(); e.print();
    std::cout << "bus1 issued " << bus1.count() << ", bus2 issued " << bus2.count() << '\n';
}

```

> Asha: Seat 1
> Bala: Seat 2
> Chitra: Seat 1
> Dev: Seat 2
> Eswar: Waiting
> bus1 issued 2, bus2 issued 2

	Static data member	Global variable	TicketOffice object
Time per ticket	O(1)	O(1)	O(1)
Space	One counter (4 bytes)	One counter	One counter per office (one int + capacity)
Encapsulation	Good: private, only BusTicket can change it	Poor: any code can change it, name clashes possible	Good: only the office's functions change it

	Static data member	Global variable	TicketOffice object
Two buses at once	No. One counter for the whole class, so both buses would share seat numbers	No, same problem	Yes. One TicketOffice per bus, as in the output above
Extra work	None	None	Every ticket constructor needs the office as argument

For one college bus the static member is the simplest and safest. If the college adds a second bus, the TicketOffice design is better.

Q6 – Friend Functions and Inline Functions (8 Marks)

Folder: Q6 Files: time_friend.cpp, time_tests.cpp, bench.cpp

6.1 Design of the comparison

Why a friend function. minutesBetween(a, b) treats both times equally. If it were a member function, one time would be the special "this" object and the other a parameter, which is awkward for a symmetric operation. A normal function needs access to the private h and m of both objects; declaring it as a friend inside the class gives exactly that access to exactly these functions, without adding public getters.

Friend vs member function.

	Member function	Friend function
How it is called	a.minutesTo(b) – on an object, with the dot	minutesBetween(a, b) – ordinary call, no object before it
this pointer	Has one	Has none
Is it in the class scope?	Yes, it is a member	No. It is only declared in the class (friend) and defined outside
Access to private data	Yes, of the object and of other objects of the same class	Yes, of every object it is given, because the class granted friendship

What inline asks the compiler to do. inline is a request to put the body of the function directly at the place of the call instead of making a real call (saving the work of passing arguments, jumping and returning). It is only a request; the compiler may refuse it, and at -O0 GCC does not inline at all. With -O2 GCC inlines small functions on its own, even without the keyword. The keyword also has a second, guaranteed meaning: the same definition may appear in several files (needed for functions defined in a header). Functions defined inside a class are automatically inline.

What would be lost if h and m were public. Encapsulation. Any code could set h = 45; m = 99; and create a time that does not exist, and there would be no single place to check it. The class could also never change its internal form (for example to store minutes since midnight) without breaking all the code that uses h and m directly.

6.2 Complete the Code

```cpp
#include <iostream>

class Time {
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1 (done): absolute difference in minutes
    int ta = a.h * 60 + a.m;
    int tb = b.h * 60 + b.m;
    return ta > tb ? ta - tb : tb - ta;
}

bool earlier(const Time& a, const Time& b)
{
    // TODO 2 (done): true if a is earlier in the day than b
    return a.h * 60 + a.m < b.h * 60 + b.m;
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Output (matches the expected output)

```
205 false
```

6.3 Correctness Check

Hand calculation: 09:45 = 9 × 60 + 45 = 585 min, 13:10 = 790 min, difference 205. 11:50 = 710 and 12:20 = 740, difference 30. 00:00 = 0 and 23:59 = 1439, difference 1439.

```
void test(const char* label, Time x, Time y, int expMin, bool expEarlier)
{
    int  got  = minutesBetween(x, y);
    bool gotE = earlier(x, y);
    std::cout << label << " -> minutes = " << got << " (expected " << expMin << "), earlier = "
              << std::boolalpha << gotE << " (expected " << expEarlier << ")  "
              << ((got == expMin && gotE == expEarlier) ? "PASS" : "FAIL") << '\n';
}

int main()
{
    test("09:45 vs 13:10 (given)       ", Time(9, 45),  Time(13, 10), 205, true);
    test("13:10 vs 09:45 (reversed)    ", Time(13, 10), Time(9, 45),  205, false);
    test("11:50 vs 12:20 (across noon) ", Time(11, 50), Time(12, 20), 30,  true);
    test("12:20 vs 11:50 (across noon) ", Time(12, 20), Time(11, 50), 30,  false);
    test("10:30 vs 10:30 (same time)   ", Time(10, 30), Time(10, 30), 0,   false);
    test("00:00 vs 23:59 (day ends)    ", Time(0, 0),   Time(23, 59), 1439,true);
}
```

```
09:45 vs 13:10 (given)       -> minutes = 205 (expected 205), earlier = true (expected true)  PASS
13:10 vs 09:45 (reversed)    -> minutes = 205 (expected 205), earlier = false (expected false)  PASS
11:50 vs 12:20 (across noon) -> minutes = 30 (expected 30), earlier = true (expected true)  PASS
12:20 vs 11:50 (across noon) -> minutes = 30 (expected 30), earlier = false (expected false)  PASS
10:30 vs 10:30 (same time)   -> minutes = 0 (expected 0), earlier = false (expected false)  PASS
00:00 vs 23:59 (day ends)    -> minutes = 1439 (expected 1439), earlier = true (expected true)  PASS
```

All six tests pass and agree with my hand calculation, including both orders across noon and the same time twice (0 minutes, `earlier` is false because a time is not earlier than itself).

6.4 Growth Investigation – timing 10,000,000 calls

I called each version 10,000,000 times on a table of 1024 different times (indexes computed with bit-masks, so the loop itself is cheap) and took the best of 7 runs. The "no inline" version is forced with `__attribute__((noinline))`, because at -O2 GCC would inline it anyway. The checksum of all four versions was identical, so they do the same work.

```cpp
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int  minutesBetweenNoInline(const Time& a, const Time& b);
    friend int  minutesBetweenInline(const Time& a, const Time& b);
    int minutesTo(const Time& o) const
    {
        int ta = h * 60 + m, tb = o.h * 60 + o.m;
        return ta > tb ? ta - tb : tb - ta;
    }
    int getH() const { return h; }
    int getM() const { return m; }
};

__attribute__((noinline)) int minutesBetweenNoInline(const Time& a, const Time& b)
{
    int ta = a.h * 60 + a.m, tb = b.h * 60 + b.m;
    return ta > tb ? ta - tb : tb - ta;
}
inline int minutesBetweenInline(const Time& a, const Time& b)
{
    int ta = a.h * 60 + a.m, tb = b.h * 60 + b.m;
    return ta > tb ? ta - tb : tb - ta;
}
inline int minutesGetters(const Time& a, const Time& b)
{
    int ta = a.getH() * 60 + a.getM(), tb = b.getH() * 60 + b.getM();
    return ta > tb ? ta - tb : tb - ta;
}
```

Version (10,000,000 calls)	-O0 time (ms)	-O2 time (ms)
Friend, noinline	59.4	14.4
Friend, inline keyword	57.6	12.6
Member function	72.9	11.4
Public getters	101.3	12.4

Explanation. At -O0 the inline keyword makes no difference (59.4 vs 57.6 ms is just noise), because the compiler does not inline when optimisation is off. The getter version is slowest (101 ms) because each of the four getter calls per comparison is a real function call. At -O2 the picture changes: the call overhead disappears, and the noinline version is about 13% slower than the inline one (14.4 vs 12.6 ms). The member and getter versions are inlined automatically and are as fast as the friend inline version (11 to 12 ms). So at -O2 the keyword matters less than the optimiser, and the difference for this tiny function is only around 2 ms per 10 million calls. The timings come from my machine; the exact milliseconds on another computer will be different, but the pattern should be the same.

6.5 Deployment Decision

I would keep the friend functions, defined inline in the header. Reasons: the operations are symmetric, so a normal function call reads naturally; h and m stay private and no getters are needed (getters would let every part of the program read the raw hours and minutes); and at -O2 they are as fast as the member and getter versions. The cost of this choice is that the friends are tightly connected to the internals of Time: if the representation changes, these two functions must be changed too, which is acceptable because there are only two and they are next to the class. If the project later wanted more operations on Time, I would reconsider and add a small public interface.

6.6 Alternative Concept and Complexity Analysis

```
// member version
int  minutesTo(const Time& o) const { int ta = h*60+m, tb = o.h*60+o.m; return ta > tb ? ta-tb : tb-ta; }
bool isEarlier(const Time& o) const { return h*60+m < o.h*60+o.m; }

// getter version
int getH() const { return h; }   int getM() const { return m; }
// then a free function minutesBetween(a,b) uses a.getH() * 60 + a.getM() ...
```

	Friend	**Member**	**Public getters**
Time per call	O(1)	O(1)	O(1)
Measured -O0 / -O2 (ms per 10M)	57.6 / 12.6	72.9 / 11.4	101.3 / 12.4
Space	O(1)	O(1)	O(1)
Encapsulation	Private data stays private; only two named functions have access	Best: only class members have access	Weakest: everybody can read h and m
Call style	minutesBetween(a, b)	a.minutesTo(b)	free function using getters

After inlining (-O2) all three have almost the same speed, so the choice should be based on encapsulation and readability, not speed.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected program: **Q4 – SavingsAccount mini-statement (shifting array).** Folder: Q7 Files: bench_history.cpp, sizes.cpp

A. Diagnose the Limitation (2 Marks)

When the 11th transaction arrives, my record() moves every stored transaction one place left. For capacity k that is k − 1 copies of a 24-byte record for every single deposit or withdrawal, so the cost grows with k. The bank's next requirement (50 transactions) makes this worse. I measured 1,000,000 transactions with the same program for different capacities:

Capacity k	Shift, -O0 (ms)	Shift, -O2 (ms)	Time per transaction at -O0
10	21.6	11.7	≈ 22 ns
50	92.2	14.0	≈ 92 ns
500	691.9	69.9	≈ 0.69 µs
5000	7134.6	2343.8	≈ 7.1 µs

The time per transaction grows almost in proportion to k (at -O0, from k = 50 upward, every 10 times more capacity gives about 7.5 to 10 times more time). The cause is a property of the design, not of the compiler: **the position of a transaction in the array depends on its age, so removing the oldest one forces every other one to move.** At -O2 the small cases look better, because the compiler turns the loop into a fast block copy, but the O(k) growth is still clearly visible from k = 500.

B. Design Objective (2 Marks)

The redesigned class must satisfy all of these, measured with the same program on the same machine:

- Recording a transaction must take constant time: for k = 5000 the time for 1,000,000 transactions must be within 2 times the time for k = 10 (at both -O0 and -O2).
- At k = 50 (the bank's new requirement) it must be at least 5 times faster than the shifting version.
- The mini statement must be identical to the original (checked with diff).

- Extra memory per account must not be more than one int.

C. Design of the Revised Solution (4 Marks)

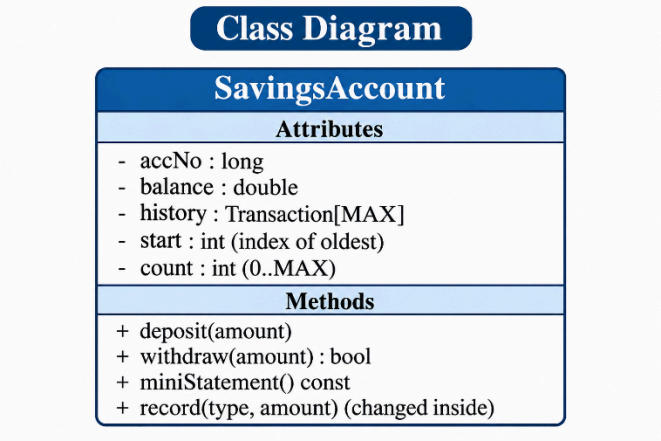
Change: keep the same fixed array but stop moving elements. Store the index `start` of the oldest transaction. A new transaction goes to slot `(start + count) % MAX` while the array is not full; when it is full, the new transaction overwrites the oldest slot and `start` moves forward by one (wrapping to 0 at the end). The statement is printed by visiting `(start + i) % MAX` for i = 0 … count − 1. The public interface (deposit, withdraw, miniStatement) is unchanged, so no calling code changes.

C++ features used: private data members, private member function `record()`, modulo arithmetic, and the same class specification as before (this is a design change inside the class, no library container).

Extra memory: one int (`start`). I checked with `sizeof`: the object is 264 bytes in both versions for k = 10 and 1224 bytes in both versions for k = 50, because the new int fits into padding bytes that already existed.

Trade-offs: the array is no longer in chronological order in memory, so the code that reads it must map the index (a bit harder to read and to debug); a modulo or compare is done on each access; and any code that wants a plain ordered array (for example to pass to another function) would have to make a copy first.

Class diagram



Pseudocode of record()

```
void add(const Transaction& t)
{
    if (count < MAX) { h[(start + count) % MAX] = t; ++count; }
    else { h[start] = t; start = (start + 1 == MAX) ? 0 : start + 1; }
}
double checksum() const { double s = 0; for (int i = 0; i < count; ++i) s += h[(start + i) % MAX].amount * (i + 1); return s; }
```

D. Predict Before Implementing (1 Mark)

The shifting version does about k − 1 copies per transaction and the circular version does one write. So I predict that the circular version's time will stay almost the same for every capacity (within about 1.5 times), while the shifting time grows roughly in proportion to k. The speed-up of circular over shift should therefore be small for k = 10 (about 3 to 8 times), around 10 to 30 times for k = 50, and more than 100 times for k = 500 and k = 5000. The measure is the time for 1,000,000 transactions.

E. Implement and Validate (2 Marks)

Both versions run the same test (1,000,000 transactions, best of 5 runs, the same data). A checksum of the final statement was equal for shift, circular and deque in every run, and the full 12-transaction statement of Q4 was identical (diff printed no difference).

```
struct Transaction { char type; double amount; double balanceAfter; };

// ORIGINAL design: shift the array left when full  -> O(MAX) per record
template <int MAX>
class ShiftHistory {
    Transaction h[MAX];
    int count = 0;
public:
    void add(const Transaction& t)
    {
        if (count == MAX) {
            for (int i = 0; i < MAX - 1; ++i) h[i] = h[i + 1];
            h[MAX - 1] = t;
        } else h[count++] = t;
    }
    double checksum() const { double s = 0; for (int i = 0; i < count; ++i) s += h[i].amount * (i + 1); return s; }
};

// REDESIGN: circular buffer with a start index -> O(1) per record
template <int MAX>
class CircularHistory {
    Transaction h[MAX];
    int start = 0, count = 0;
public:
    void add(const Transaction& t)
    {
        if (count < MAX) { h[(start + count) % MAX] = t; ++count; }
        else { h[start] = t; start = (start + 1 == MAX) ? 0 : start + 1; }
    }
    double checksum() const { double s = 0; for (int i = 0; i < count; ++i) s += h[(start + i) % MAX].amount * (i + 1); return s; }
};

// ALTERNATIVE: std::deque (push_back + pop_front)
template <int MAX>
class DequeHistory {
    std::deque<Transaction> h;
public:
    void add(const Transaction& t) { h.push_back(t); if ((int)h.size() > MAX) h.pop_front(); }
    double checksum() const { double s = 0; int i = 0; for (auto& t : h) s += t.amount * (++i); return s; }
};
```

Original (shift) → Redesigned (circular), with deque for comparison

k	Shift O0 (ms)	Circular O0 (ms)	Shift O2 (ms)	Circular O2 (ms)	Deque O2 (ms)	Speed-up O0 / O2
10	21.6	3.87	11.7	1.57	10.4	5.6× / 7.5×
50	92.2	3.47	14.0	0.93	6.6	26.6× / 15.1×
500	691.9	3.22	69.9	1.05	7.8	215× / 66×
5000	7134.6	3.73	2343.8	1.11	8.0	1913× / 2120×

Result against my prediction: the circular time stayed flat (3.2 to 3.9 ms at -O0 and 0.9 to 1.6 ms at -O2 across all capacities), as predicted. The speed-ups were also in the predicted order for k = 10, 50 and 5000. The one place where I was off is k = 500 at -O2: I expected more than 100 times but got 66 times, probably because at -O2 the compiler turns the shifting loop into fast block copies, which are cheap while the data is small; the shifting cost then jumps at k = 5000, where the 120 KB of records no longer fits comfortably in the fast cache.

F. Defend the Final Design (2 Marks)

Was the objective achieved? Yes, all four parts. (1) Circular time for k = 5000 is 3.73 ms at -O0 versus 3.87 ms for k = 10, and 1.11 ms versus 1.57 ms at -O2, so it is well inside the 2× limit. (2) At k = 50 it is 26.6 times faster at -O0 and 15.1 times at -O2, above the 5× target. (3) The statement is identical. (4) Extra memory is one int, which cost 0 extra bytes because of padding.

New costs: harder-to-read index logic, a possible off-by-one bug in the wrap-around, and the data in memory is not in time order.

Where the original may still be preferable: for the original k = 10 the shifting version takes only about 22 ns (-O0) or 12 ns (-O2) per transaction, which is far too small to notice at an ATM. Its code is the simplest to read and test, and the array is always in order, so a statement can be printed or passed on directly. So if the capacity was never going to change, the original is good enough. The redesign is worth it because the bank's requirement is already changing (10 → 50) and may change again, and because the cost no longer depends on that decision.

About std::deque: it also gives O(1) per transaction and needs no index arithmetic, but in my measurement it was about 6 to 8 times slower than the circular buffer (6.6 to 10.4 ms at -O2 against

0.9 to 1.6 ms), because it allocates memory in blocks on the heap and has an extra level of indirection. It would be the better choice if the number of stored transactions had to be changed at run time.